

Rajalakshmi Engineering College

Name: Naren S
Email: 241901066@rajalakshmi.edu.in
Roll no: 241901066
Phone: 6382463115
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units 4 per unit For the next 100 units (51–150) 6 per unit
For units above 150 8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

Answer

```
import java.util.Scanner;
```

```
class Customer {
    private int customerId;
    private String customerName;
    private double unitsConsumed;

    public Customer(int customerId, String customerName, double
unitsConsumed) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.unitsConsumed = unitsConsumed;
    }

    public double calculateBill() {
        double bill = 0;
        double units = unitsConsumed;

        if (units <= 50) {
            bill = units * 4;
        } else if (units <= 150) {
            bill = (50 * 4) + ((units - 50) * 6);
        } else {
            bill = (50 * 4) + (100 * 6) + ((units - 150) * 8);
        }

        if (bill > 2000) {
            bill = bill - (bill * 0.15);
        }

        return bill;
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getCustomerName() {
        return customerName;
    }
}

public class Main {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = Integer.parseInt(sc.nextLine());

    for (int i = 0; i < n; i++) {
        int customerId = Integer.parseInt(sc.nextLine());
        String customerName = sc.nextLine();
        double unitsConsumed = Double.parseDouble(sc.nextLine());

        Customer customer = new Customer(customerId, customerName,
unitsConsumed);
        double finalBill = customer.calculateBill();

        System.out.printf("Customer ID: %d Customer Name: %s Final Bill: %.1f",
customer.getCustomerId(), customer.getCustomerName(), finalBill);
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed (double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per liter For the next 500 liters (501–1000) 3 per liter
For liters above 1000 5 per liter If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize

customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

Answer

```
import java.util.Scanner;
```

```
class Customer {
    private int customerId;
    private String customerName;
    private double litersConsumed;

    public Customer(int customerId, String customerName, double
    litersConsumed) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.litersConsumed = litersConsumed;
    }

    public double calculateBill() {
        double bill = 0;
        double liters = litersConsumed;

        if (liters <= 500) {
            bill = liters * 2;
        } else if (liters <= 1000) {
            bill = (500 * 2) + ((liters - 500) * 3);
        } else {
            bill = (500 * 2) + (500 * 3) + ((liters - 1000) * 5);
        }

        if (bill > 3000) {
            bill = bill - (bill * 0.10);
        }

        return bill;
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getCustomerName() {
        return customerName;
    }
}

public class Main {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int n = Integer.parseInt(sc.nextLine());

for (int i = 0; i < n; i++) {
    int customerId = Integer.parseInt(sc.nextLine());
    String customerName = sc.nextLine();
    double litersConsumed = Double.parseDouble(sc.nextLine());

    Customer customer = new Customer(customerId, customerName,
    litersConsumed);
    double finalBill = customer.calculateBill();

    System.out.printf("Customer ID: %d Customer Name: %s Final Bill: %.1f",
    customer.getCustomerId(), customer.getCustomerName(), finalBill);
}
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are working as a developer for CityMobile, which wants to build a basic mobile data usage management system.

Each customer has:

A Customer ID (integer) A Customer Name (string) An Initial Data Balance (in GB, double)

The company allows two types of operations:

Recharge – increases the data balance. Usage – decreases the data balance only if enough data is available.

If the usage amount is greater than the available data balance, the usage should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details after all operations.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Initial Data Balance (double).
- The next line contains the Recharge Amount in GB (double).
- The next line contains the Usage Amount in GB (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Data Balance: <final_data_balance> GB (The final balance must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Ravi Kumar

5.0

2.0

3.0

Output: Customer ID: 1234

Customer Name: Ravi Kumar
Final Data Balance: 4.0 GB

Answer

```
import java.util.Scanner;
```

```
class Customer {  
    private int customerId;  
    private String customerName;  
    private double dataBalance;  
  
    public Customer(int customerId, String customerName, double dataBalance) {  
        this.customerId = customerId;  
        this.customerName = customerName;  
        this.dataBalance = dataBalance;  
    }  
  
    public void recharge(double amount) {  
        if (amount >= 0) {  
            dataBalance += amount;  
        }  
    }  
  
    public void useData(double amount) {  
        if (amount <= dataBalance) {  
            dataBalance -= amount;  
        }  
    }  
  
    public int getCustomerId() {  
        return customerId;  
    }  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public double getDataBalance() {  
        return dataBalance;  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());

        for (int i = 0; i < n; i++) {
            int customerId = Integer.parseInt(sc.nextLine());
            String customerName = sc.nextLine();
            double initialBalance = Double.parseDouble(sc.nextLine());
            double rechargeAmount = Double.parseDouble(sc.nextLine());
            double usageAmount = Double.parseDouble(sc.nextLine());

            Customer customer = new Customer(customerId, customerName,
            initialBalance);
            customer.recharge(rechargeAmount);
            customer.useData(usageAmount);

            System.out.printf("Customer ID: %d Customer Name: %s Final Data
            Balance: %.1f GB",
                customer.getCustomerId(), customer.getCustomerName(),
            customer.getDataBalance());
        }
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Anjali is working as a developer for the City Basketball Association, which wants to build a system to track and find the top scorer among basketball players.

Each player's record has:

Player ID (integer) Player Name (string) An array of points scored in 5 matches (integers)

The system must calculate:

The total score of each player (sum of all match points). Identify the highest scorer among all players. If two or more players have the same total score, the one with the lower Player ID is considered the top scorer.

Anjali has been asked to implement this system using:

A class with attributes for player details. A constructor to initialize player details. Getter and Setter methods to retrieve and update player details if required. A method to calculate the total score. Objects of the class to represent players.

Finally, display each player's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N (number of players).

For each player:

- The next line contains the Player ID (integer).
- The following line contains the Player Name (string).
- The next line contains 5 integers separated by spaces (points scored in 5 matches).

Output Format

For each player the output prints the following details:

- Player ID: <player_id>
- Player Name: <player_name>
- Total Score: <total_score>

Finally, print "Top Scorer: <player_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

10 20 30 40 50

Output: Player ID: 1001

Player Name: Ravi Kumar

Total Score: 150

Top Scorer: Ravi Kumar with 150 points

Answer

```
import java.util.Scanner;
```

```
class Player {  
    private int playerId;  
    private String playerName;  
    private int[] scores;  
  
    public Player(int playerId, String playerName, int[] scores) {  
        this.playerId = playerId;  
        this.playerName = playerName;  
        this.scores = scores;  
    }  
  
    public int getPlayerId() {  
        return playerId;  
    }  
  
    public String getPlayerName() {  
        return playerName;  
    }  
  
    public int getTotalScore() {  
        int total = 0;  
        for (int score : scores) {  
            total += score;  
        }  
        return total;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int n = Integer.parseInt(sc.nextLine());

Player topScorer = null;
int highestScore = -1;

Player[] players = new Player[n];

for (int i = 0; i < n; i++) {
    int playerId = Integer.parseInt(sc.nextLine());
    String playerName = sc.nextLine();
    String[] scoreTokens = sc.nextLine().split(" ");
    int[] scores = new int[5];
    for (int j = 0; j < 5; j++) {
        scores[j] = Integer.parseInt(scoreTokens[j]);
    }

    Player player = new Player(playerId, playerName, scores);
    players[i] = player;

    int totalScore = player.getTotalScore();

    System.out.printf("Player ID: %d Player Name: %s Total Score: %d",
        player.getPlayerId(), player.getPlayerName(), totalScore);

    if (totalScore > highestScore ||
        (totalScore == highestScore && player.getPlayerId() <
        topScorer.getPlayerId())) {
        highestScore = totalScore;
        topScorer = player;
    }
}

System.out.printf("Top Scorer: %s with %d points",
    topScorer.getPlayerName(), topScorer.getTotalScore());
}

```

Status : Correct

Marks : 10/10